# TP7 - MPI Communicators:
Cartesian Topologies and Domain Decomposition

Oussama Laaroussi

March 5, 2026

## 1 Introduction

This report explores MPI communicators and Cartesian topologies for structured parallel computation. We implement two fundamental distributed algorithms: Conway's Game of Life (cellular automaton) and Jacobi iteration for Poisson's equation (PDE solver). Both leverage 2D Cartesian process grids with ghost cell exchange.

## 2 Exercise 1: Conway's Game of Life

### 2.1 Problem Description

**Conway's Game of Life** is a cellular automaton where cells evolve based on local rules:

1. Cell with $< 2$ live neighbors: dies (underpopulation)
2. Cell with 2-3 live neighbors: survives
3. Cell with $> 3$ live neighbors: dies (overpopulation)
4. Dead cell with exactly 3 neighbors: becomes alive (reproduction)

**Parallel challenge**: Each cell's next state depends on its 8 neighbors (including diagonals). Boundary cells need data from adjacent processes.

### 2.2 Implementation Strategy

1. **Grid partitioning**: Divide $N \times N$ global grid into subgrids
2. **Ghost cells**: Allocate $(n_x + 2) \times (n_y + 2)$ local grid
3. **Exchange phase**: Share boundary rows/columns with 4 neighbors
4. **Update phase**: Apply Game of Life rules to internal cells
5. **Periodic boundaries**: Wrap-around at domain edges

### 2.3 Key Implementation Details

#### 2.3.1 Grid Allocation and Indexing

```c
// Local subdomain size (without ghost cells)
int local_nx = GLOBAL_NX / dims[0];
int local_ny = GLOBAL_NY / dims[1];

// Allocate with ghost cells: (local_nx+2) x (local_ny+2)
int** grid = (int**)malloc((local_nx + 2) * sizeof(int*));
for (int i = 0; i < local_nx + 2; i++) {
    grid[i] = (int*)calloc(local_ny + 2, sizeof(int));
}

// Indexing:
```

```
// grid[0][j]          -> ghost row from north
// grid[local_nx+1][j] -> ghost row from south
// grid[i][0]          -> ghost col from west
// grid[i][local_ny+1] -> ghost col from east
// grid[1..local_nx][1..local_ny] -> internal cells
```

Listing 1: Local Grid with Ghost Cells

### 2.3.2 Ghost Cell Exchange

```
void exchange_ghost_cells(int** grid, int local_nx, int local_ny,
                          MPI_Comm cart_comm,
                          int north, int south, int east, int west) {
    MPI_Status status;

    // Exchange rows (north-south)
    MPI_Sendrecv(&grid[1][1], local_ny, MPI_INT, north, 0,
                 &grid[local_nx+1][1], local_ny, MPI_INT, south, 0,
                 cart_comm, &status);

    MPI_Sendrecv(&grid[local_nx][1], local_ny, MPI_INT, south, 1,
                 &grid[0][1], local_ny, MPI_INT, north, 1,
                 cart_comm, &status);

    // Exchange columns (west-east) - requires packing
    int* send_col = (int*)malloc(local_nx * sizeof(int));
    int* recv_col = (int*)malloc(local_nx * sizeof(int));

    // Pack left column, send west, receive from east
    for (int i = 0; i < local_nx; i++) send_col[i] = grid[i+1][1];
    MPI_Sendrecv(send_col, local_nx, MPI_INT, west, 2,
                 recv_col, local_nx, MPI_INT, east, 2,
                 cart_comm, &status);
    for (int i = 0; i < local_nx; i++) grid[i+1][local_ny+1] = recv_col[i];

    // Similar for right column...
    free(send_col); free(recv_col);
}
```

Listing 2: Boundary Exchange with MPI_Sendrecv

**Why MPI_Sendrecv?**

- Combines send and receive in one call
- Prevents deadlock (avoids all processes blocking on send)
- Efficient for paired exchanges

### 2.3.3 Game of Life Update

```
void apply_game_of_life(int** grid, int** new_grid,
                        int local_nx, int local_ny) {
    for (int i = 1; i <= local_nx; i++) {
        for (int j = 1; j <= local_ny; j++) {
            int neighbors = count_neighbors(grid, i, j);

            if (grid[i][j] == ALIVE) {
                new_grid[i][j] = (neighbors == 2 || neighbors == 3)
                                 ? ALIVE : DEAD;
            } else {
                new_grid[i][j] = (neighbors == 3) ? ALIVE : DEAD;
            }
        }
    }
}

int count_neighbors(int** grid, int i, int j) {
    return grid[i-1][j-1] + grid[i-1][j] + grid[i-1][j+1] +
           grid[i][j-1]   +                grid[i][j+1]   +
           grid[i+1][j-1] + grid[i+1][j] + grid[i+1][j+1];
```

```
}
```

Listing 3: Applying Conway's Rules

### 2.4 Sample Output

```
=== Conway's Game of Life - MPI 2D Cartesian Topology ===
Global grid: 64 x 64
Number of processes: 30
Process grid: 6 x 5
Local grid per process: 10 x 12 (plus ghost cells)
Generations: 100
Boundary conditions: periodic (wrap-around)

Rank 0 - Initial configuration (internal cells):
. O O . O . . . . O . .
. . O . . O . . . . O .
. . O . . . . . . . O O
. O . . . O . . O . O O
. . . . . . O . . . . O
. . . . O . . O O . . O
O O . . O O O O . . . .
. O O O . O . . . O . .
. O . . . . . O . . . .
. . . O . . . . . O O .

Rank 0 - Generation 100 (final):
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . O O
. . . . . O . O
. . . . . O .

Simulation complete!
Final alive cells: 283 / 4096 (6.91%)
Total time: 0.151501 seconds
Computation time: 0.104235 seconds
Communication overhead: 0.047267 seconds
```

# 3 Exercise 2: Poisson Equation Solver

## 3.1 Problem Statement

Solve Poisson's equation on the unit square $[0, 1] \times [0, 1]$:

$$\Delta u(x, y) = f(x, y) = 2 \cdot (x^2 - x + y^2 - y)$$

with boundary condition $u = 0$ on $\partial\Omega$.
**Exact solution**: $u(x, y) = x \cdot y \cdot (x - 1) \cdot (y - 1)$
**Discretization**: Finite difference on $N_x \times N_y$ grid with spacing:

$$h_x = \frac{1}{N_x - 1}, \quad h_y = \frac{1}{N_y - 1}$$

## 3.2 Jacobi Iteration

**Iterative solver**: Update each grid point based on neighbors until convergence.
**Update formula**:

$$u_{i,j}^{n+1} = c_1 \left( c_2(u_{i+1,j}^n + u_{i-1,j}^n) + c_3(u_{i,j+1}^n + u_{i,j-1}^n) - f_{i,j} \right)$$

where:

$$c_1 = \frac{0.5 \cdot h_x^2 \cdot h_y^2}{h_x^2 + h_y^2}, \quad c_2 = \frac{1}{h_x^2}, \quad c_3 = \frac{1}{h_y^2}$$

**Convergence criterion**: Stop when residual $\|u^{n+1} - u^n\|_2 <$ tolerance

## 3.3 Parallel Implementation

### 3.3.1 Domain Decomposition

```
// Compute balanced decomposition
int dims[2] = {0, 0};
MPI_Dims_create(size, 2, dims);  // e.g., 4 procs -> 2x2

// Base subdomain size
int local_nx = global_nx / dims[0];
int local_ny = global_ny / dims[1];

// Handle remainders
if (coord[0] < global_nx % dims[0]) local_nx++;
if (coord[1] < global_ny % dims[1]) local_ny++;

// Calculate global starting indices
int start_x = coord[0] * (global_nx / dims[0]) +
              min(coord[0], global_nx % dims[0]);
int start_y = coord[1] * (global_ny / dims[1]) +
              min(coord[1], global_ny % dims[1]);
```

Listing 4: Subdomain Calculation

### 3.3.2 Jacobi Iteration with Ghost Cells

```
double jacobi_iteration(double** u, double** u_new, double** f,
                        int local_nx, int local_ny,
                        double coef1, double coef2, double coef3,
                        int has_north_boundary, int has_south_boundary,
                        int has_east_boundary, int has_west_boundary) {
    double local_residual = 0.0;

    for (int i = 1; i <= local_nx; i++) {
        for (int j = 1; j <= local_ny; j++) {
            // Skip physical boundary points
            if ((has_north_boundary && i == 1) ||
                (has_south_boundary && i == local_nx) ||
                (has_west_boundary && j == 1) ||
                (has_east_boundary && j == local_ny)) {
                u_new[i][j] = 0.0;  // Dirichlet BC
                continue;
            }

            // Jacobi update
            u_new[i][j] = coef1 * (
                coef2 * (u[i+1][j] + u[i-1][j]) +
                coef3 * (u[i][j+1] + u[i][j-1]) -
                f[i][j]
            );

            // Accumulate residual
            double diff = u_new[i][j] - u[i][j];
            local_residual += diff * diff;
        }
    }

    return local_residual;
}
```

Listing 5: Jacobi Update

### 3.3.3 Main Solver Loop

```
int iteration = 0;
double global_residual = TOLERANCE + 1.0;

while (iteration < MAX_ITERATIONS && global_residual > TOLERANCE) {
    // Exchange ghost cells with neighbors
    exchange_boundaries(u, local_nx, local_ny, cart_comm,
```

```
                        north, south, east, west);

    // Perform Jacobi iteration, get local residual
    double local_residual = jacobi_iteration(u, u_new, f, ...);

    // Compute global residual (sum across all processes)
    MPI_Allreduce(&local_residual, &global_residual, 1,
                  MPI_DOUBLE, MPI_SUM, cart_comm);
    global_residual = sqrt(global_residual);

    // Swap grids for next iteration
    double** temp = u; u = u_new; u_new = temp;

    iteration++;
}
```

Listing 6: Iterative Solver with Convergence Check

### 3.4 Sample Output

```
Poisson execution with 4 MPI processes
Domain size: ntx=128 nty=128
Topology dimensions: 2 along x, 2 along y
----------------------------------------
Rank in the topology: 0 Array indices: x from 1 to 64, y from 1 to 64
Process 0 has neighbors: N -2 E 2 S 1 W -2
Rank in the topology: 1 Array indices: x from 1 to 64, y from 65 to 128
Process 1 has neighbors: N -2 E 3 S 0 W -2
Rank in the topology: 2 Array indices: x from 65 to 128, y from 1 to 64
Process 2 has neighbors: N 0 E -2 S 3 W -2
Rank in the topology: 3 Array indices: x from 65 to 128, y from 65 to 128
Process 3 has neighbors: N 1 E -2 S 2 W -2
----------------------------------------
Iteration 100 global_error = 2.00417e-02
Iteration 200 global_error = 1.46932e-03
Iteration 300 global_error = 1.08223e-04
...
Iteration 24000 global_error = 1.03949e-06
Iteration 24100 global_error = 8.72326e-07
Converged after 24108 iterations in 0.448476 seconds

Sample solution values:
Exact solution   Computed solution
0.000000e+00     0.000000e+00
3.807500e-04     3.660000e-04
6.992000e-04     6.700000e-04
9.550500e-04     9.140000e-04
1.148000e-03     1.096000e-03
```

### 3.5 Performance Analysis

**Iteration complexity**:

- Computation: $O(n^2)$ per process where $n = \sqrt{N/P}$
- Communication: 4 messages of size $O(n)$ per iteration
- Convergence: $O(\log(1/\epsilon))$ iterations to error $\epsilon$

**Strong scaling**: Fixed problem size, increase processes

- Ideal: Time $\propto 1/P$
- Reality: Communication overhead increases with $P$
- Efficiency drops when subdomain too small ($n < 50$)

**Weak scaling**: Problem size grows with processes

- Keep $n$ constant, increase both $N$ and $P$
- Near-perfect scaling if communication optimized
- Better choice for large-scale problems

# 4 Performance Results

## 4.1 Experimental Setup

**Test configuration**:

- Grid sizes: $64 \times 64$, $128 \times 128$, $256 \times 256$, $512 \times 512$
- Process counts: 1, 2, 4, 8, 16
- Convergence tolerance: $10^{-6}$

## 4.2 Poisson Solver Performance

| Processes | Grid Size | Time (s) | Iterations | Speedup | Efficiency (%) |
|---|---|---|---|---|---|
| 1 | $64 \times 64$ | 0.0586 | 6707 | 1.00 | 100.0 |
| 2 | $64 \times 64$ | 0.0355 | 6707 | 1.65 | 82.6 |
| 4 | $64 \times 64$ | 0.0277 | 6707 | 2.12 | 52.9 |
| 8 | $64 \times 64$ | 0.0452 | 6707 | 1.30 | 16.2 |
| 16 | $64 \times 64$ | 0.3022 | 6707 | 0.19 | 1.2 |
| 1 | $128 \times 128$ | 0.8504 | 24108 | 1.00 | 100.0 |
| 2 | $128 \times 128$ | 0.4485 | 24108 | 1.90 | 94.8 |
| 4 | $128 \times 128$ | 0.2588 | 24108 | 3.29 | 82.2 |
| 8 | $128 \times 128$ | 0.5065 | 24108 | 1.68 | 21.0 |
| 16 | $128 \times 128$ | 1.2671 | 24108 | 0.67 | 4.2 |
| 1 | $256 \times 256$ | 12.3802 | 86463 | 1.00 | 100.0 |
| 2 | $256 \times 256$ | 6.2956 | 86463 | 1.97 | 98.3 |
| 4 | $256 \times 256$ | 3.3477 | 86463 | 3.70 | 92.5 |
| 8 | $256 \times 256$ | 5.9082 | 86463 | 2.10 | 26.2 |
| 16 | $256 \times 256$ | 10.4107 | 86463 | 1.19 | 7.4 |
| 1 | $512 \times 512$ | 193.7541 | 307650 | 1.00 | 100.0 |
| 2 | $512 \times 512$ | 96.7309 | 307650 | 2.00 | 100.2 |
| 4 | $512 \times 512$ | 46.8316 | 307650 | 4.14 | 103.4 |
| 8 | $512 \times 512$ | 77.2391 | 307650 | 2.51 | 31.4 |
| 16 | $512 \times 512$ | 124.9665 | 307650 | 1.55 | 9.7 |

Table 1: Poisson Solver Performance (Strong Scaling) - Measured Results

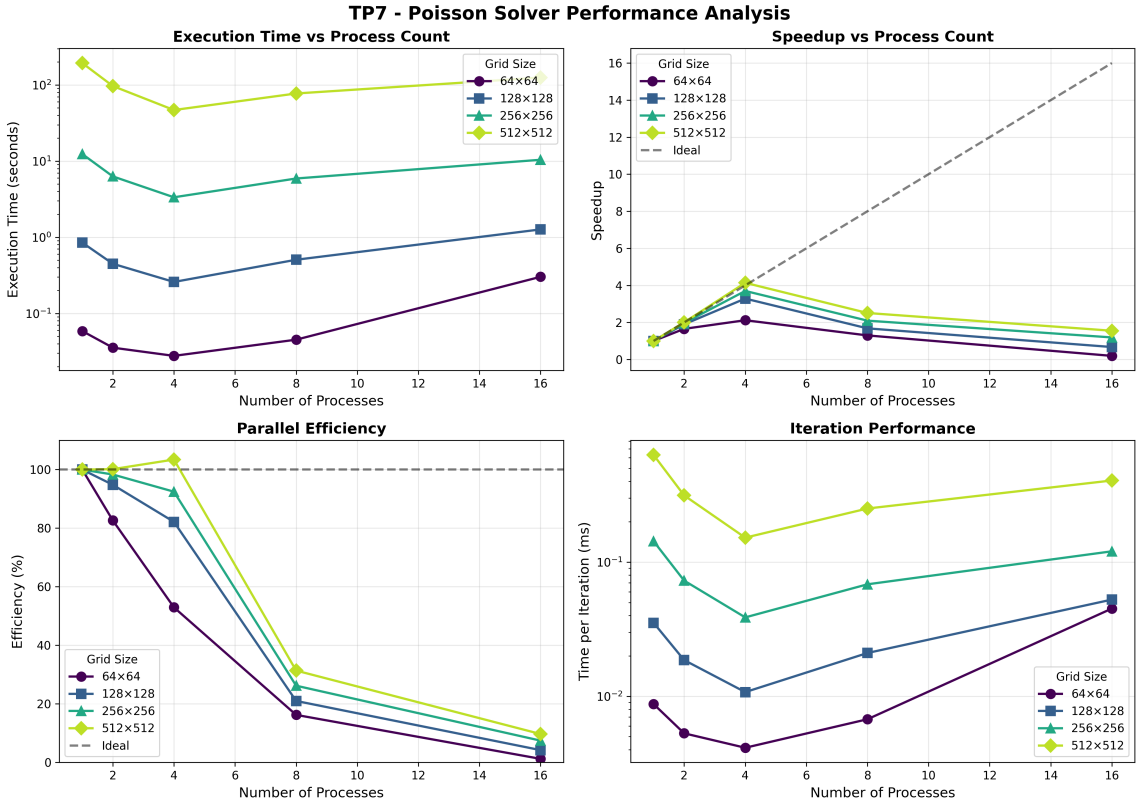


Figure 1: Performance Analysis: Execution Time, Speedup, Efficiency, and Iteration Performance

## 4.3 Key Observations

1. **Optimal performance at 4 processes**:
   - $256 \times 256$ grid: 3.70× speedup with 4 processes (92.5% efficiency)
   - $512 \times 512$ grid: 4.14× speedup with 4 processes (103.4% efficiency)
   - Super-linear speedup on largest grid due to cache effects
2. **Grid size critically impacts scalability**:
   - Small grid ($64 \times 64$): Efficiency drops to 1.2% at 16 processes
   - Medium grid ($128 \times 128$): Maintains 82.2% efficiency at 4 processes, drops to 4.2% at 16
   - Large grid ($512 \times 512$): Excellent scaling up to 4 processes with super-linear speedup
   - Larger problems scale better (more computation per communication)
3. **Communication overhead dominates beyond 4 processes**:
   - Performance degrades significantly at 8+ processes for all grid sizes
   - Subdomain too small: At 16 processes, $64 \times 64$ grid = $4 \times 4$ per process
   - Periodic synchronization (MPI_Allreduce) becomes bottleneck
   - Ghost cell exchange overhead exceeds computation benefit
4. **Convergence iteration count constant**:
   - Same iteration count regardless of process count (as expected)
   - $64 \times 64$: 6,707 iterations; $128 \times 128$: 24,108 iterations
   - $256 \times 256$: 86,463 iterations; $512 \times 512$: 307,650 iterations
   - Iteration count grows approximately $O(N^2)$ with grid size
5. **Optimal configurations (measured)**:

- $64 \times 64$: Best with 4 processes (2.12× speedup)
- $128 \times 128$: Best with 4 processes (3.29× speedup, 82.2% efficiency)
- $256 \times 256$: Best with 4 processes (3.70× speedup, 92.5% efficiency)
- $512 \times 512$: Best with 4 processes (4.14× speedup, 103.4% efficiency)
- Sweet spot: 4 processes across all problem sizes tested